

UNIVERSIDAD SANTIAGO MARIÑO

Escuela de Ingeniería

TEORÍA MODERNA DE CONTROL

SISTEMA DE LLENADO DE FLUJO CONTINUO CON CONTROL AUTOMÁTICO

RESUMEN TÉCNICO HIPER-DETALLADO

Proyecto Segundo – Versión 2.0

Autor: Neil Edickson Suarez Arevalo (NeilsAraquitec)

Co-Autor: Jose Fabien Salas Garcia

Junio 2026

0. RESUMEN EJECUTIVO

El presente documento constituye el resumen técnico hiper-detallado del proyecto **Sistema de Llenado de Flujo Continuo con Control Automático**, desarrollado en el marco de la asignatura *Teoría Moderna de Control* de la Universidad Santiago Mariño. El sistema integra dos microcontroladores (Arduino Uno y ESP32), dos sensores de naturaleza distinta (ultrasónico HC-SR04 y caudalímetro GFS401), dos bombas de agua controladas por relés, y una interfaz web embebida accesible por WiFi.

El objetivo principal es demostrar la aplicación práctica de la teoría moderna de control sobre una planta real, incluyendo: modelado matemático por funciones de transferencia y representación en espacio de estados, análisis de controlabilidad y observabilidad, diseño de controlador óptimo LQR, e implementación de lógica de control por histéresis en hardware embebido de bajo costo.

Parámetro	Valor
Tiempo de subida (llenado 0 → 80%)	≈ 60–90 s
Sobrepaso	Mínimo (banda de histéresis)
Error en estado estacionario	0% (sistema tipo 1)
Oscilación en estado estable	±5% (banda histéresis)
Frecuencia de muestreo	1 Hz (serial) / 5 Hz (HC-SR04)
Caudal medio en llenado	≈ 2.5 L/min
Ciclos ON/OFF en transitorio	5–8 ciclos
Cobertura de firmware	Arduino (194 líneas) + ESP32 (331 líneas)
Simulación Python	3 escenarios, 615 líneas
Análisis MATLAB	116 líneas, Toolbox de Control

TABLA DE CONTENIDOS

1. Descripción General del Sistema
2. Arquitectura de Hardware
 - 2.1 Microcontroladores
 - 2.2 Sensores
 - 2.3 Actuadores
 - 2.4 Comunicación y Niveles Lógicos
 - 2.5 Configuración de Pines
3. Protocolo de Comunicación
4. Interfaz Web (HMI)
5. Modelado Matemático
 - 5.1 Funciones de Transferencia
 - 5.2 Representación en Espacio de Estados
 - 5.3 Análisis de Controlabilidad y Observabilidad
 - 5.4 Análisis de Estabilidad
6. Algoritmo de Control
 - 6.1 Control por Histéresis (Implementado)
 - 6.2 Control LQR Óptimo (Propuesto)
7. Firmware: Arduino Uno
8. Firmware: ESP32
9. Simulaciones
 - 9.1 Simulación Python (3 Escenarios)
 - 9.2 Script MATLAB / Simulink
10. Parámetros de Calibración
11. Estructura de Documentación
12. Lista de Materiales y Costos
13. Guía de Resolución de Problemas
14. Secuencia de Operación
15. Mejoras Futuras
16. Referencias Bibliográficas

1. DESCRIPCIÓN GENERAL DEL SISTEMA

Nombre del Proyecto: Sistema de Llenado de Flujo Continuo con Control Automático | **Versión:** 2.0 | **Idioma:** Español | **Fecha:** Junio 2026 | **Institución:** Universidad Santiago Mariño

El sistema controla automáticamente el nivel de llenado de un depósito de agua utilizando dos microcontroladores que trabajan en red: el **Arduino Uno** se encarga de toda la lógica de control, la adquisición de datos de los sensores y la actuación sobre las bombas; el **ESP32** actúa como interfaz IoT, publicando un servidor web embebido accesible desde cualquier dispositivo con navegador en la misma red WiFi.

La arquitectura separa claramente las responsabilidades: el control en tiempo real queda en el Arduino (ciclo de control de 200 ms) y la capa de comunicación/HMI en el ESP32. Ambos se comunican por UART serie a 9600 baudios, con adaptación de niveles lógicos (5V Arduino → 3.3V ESP32) mediante divisor de tensión resistivo.

Objetivos del Proyecto

- Aplicar la teoría moderna de control (espacio de estados, LQR) sobre una planta física real.
- Implementar un sistema embebido de control automático de nivel con sensor ultrasónico.
- Integrar medición de caudal mediante caudalímetro GFS401 por conteo de pulsos.
- Desarrollar una interfaz HMI web accesible por WiFi sin necesidad de infraestructura externa.
- Validar el modelo matemático de la planta mediante simulación Python y análisis MATLAB.
- Documentar el proyecto con estándares académicos y reproducibilidad total.

2. ARQUITECTURA DE HARDWARE

2.1 Microcontroladores

Característica	Arduino Uno (ATmega328P)	ESP32-DevKit
CPU / Frecuencia	ATmega328P / 16 MHz	Xtensa LX6 / 240 MHz
Voltaje de operación	5V	3.3V
Flash / RAM	32 KB / 2 KB SRAM	4 MB / 520 KB SRAM
GPIO digitales	14 (6 PWM)	34
ADC	6 canales (10-bit)	18 canales (12-bit)
Comunicación	UART, SPI, I2C, USB	UART, SPI, I2C, WiFi 802.11, BT
Modo WiFi	No integrado	Access Point (AP) 2.4 GHz
Función en el sistema	Control principal, sensores, relés	Servidor web, UART bridge

2.2 Sensores

Característica	HC-SR04 (Nivel)	GFS401 (Caudal)
Magnitud medida	Distancia / nivel (%)	Caudal (L/min)
Principio	Eco ultrasónico (40 kHz)	Efecto Hall (paletas giratorias)
Rango de medición	2–400 cm	1–30 L/min
Resolución	0.3 cm	≈ 0.13 L/min (1 pulso)
Factor de calibración	Distancia → %: configurable	7.5 pulsos / L/min
Voltaje de alimentación	5V DC	5V DC
Señal de salida	ECHO (pulso ancho)	Tren de pulsos digitales
Pin Arduino	TRIG: 9, ECHO: 8	Pin 2 (INT0, flanco bajada)
Ruido modelado	Gaussiano $\sigma = 0.4\%$	Error de cuantización

2.3 Actuadores

Componente	Bomba 1 (Llenado)	Bomba 2 (Drenaje)
Tipo	Bomba DC sumergible	Bomba DC sumergible / circulación
Voltaje	5V o 12V DC	5V o 12V DC
Control	Relé 2 canales (activo LOW)	Relé 2 canales (activo LOW)

Componente	Bomba 1 (Llenado)	Bomba 2 (Drenaje)
Pin Arduino relé	Pin 6	Pin 7
Estado activo	Pin LOW → Relé activado → Bomba ON	Igual
Estado inactivo	Pin HIGH → Relé libre → Bomba OFF	Igual
Función en AUTO	Controlada automáticamente	Sin control AUTO (manual)

2.4 Comunicación y Niveles Lógicos

La comunicación bidireccional entre Arduino Uno (5V) y ESP32 (3.3V) se realiza por UART mediante la librería **SoftwareSerial** en el Arduino (pines 10 y 11). Como el ESP32 no tolera señales de 5V en sus GPIO, se implementa un **divisor de tensión resistivo** en la línea TX del Arduino (Arduino-TX → ESP32-RX): resistencias R1=10 kΩ y R2=20 kΩ reducen 5V a 3.3V exactos. La línea opuesta (ESP32-TX → Arduino-RX) no requiere adaptación ya que el Arduino reconoce 3.3V como nivel lógico alto.

Parámetro	Valor
Velocidad	9600 baudios, 8N1
Pines Arduino	RX=10, TX=11 (SoftwareSerial)
Pines ESP32	RX=GPIO16 (Serial2), TX=GPIO17
Adaptación de nivel	Divisor: R1=10kΩ, R2=20kΩ → 3.33V
Período de transmisión	Cada 1000 ms (T_ENVIO_MS)
Timeout ESP32 → Arduino	5000 ms (detecta desconexión)
Debug USB (Arduino)	115200 baudios (Monitor Serie)

2.5 Configuración Completa de Pines

Arduino Uno:

Pin	Función	Tipo	Señal / Descripción
2	GFS401	Input (INT0)	Pulsos caudalímetro (flanco bajada)
6	Relé Bomba 1	Output	Active LOW → Bomba llenado
7	Relé Bomba 2	Output	Active LOW → Bomba drenaje
8	HC-SR04 ECHO	Input	Pulso de retorno (duración → distancia)
9	HC-SR04 TRIG	Output	Pulso de 10 μs para disparar medición
10	SoftwareSerial RX	Input	Recibe comandos desde ESP32
11	SoftwareSerial TX	Output	Envía estado al ESP32 (→ div. tensión)

ESP32 DevKit:

GPIO	Función	Tipo	Señal / Descripción
16	Serial2 RX	Input	Recibe datos de estado desde Arduino
17	Serial2 TX	Output	Envía comandos al Arduino

3. PROTOCOLO DE COMUNICACIÓN

Se utiliza un protocolo ASCII propio, orientado a texto, con pares clave:valor separados por comas y terminados en salto de línea. Esto facilita el depurado visual y la compatibilidad con el Monitor Serie del Arduino IDE.

Trama Arduino → ESP32 (cada 1 s)

Formato:

N:{nivel},Q:{caudal},B1:{bomba1},B2:{bomba2},SP:{setpoint},M:{modo},A:{alarma}

Campo	Tipo	Rango	Descripción
N	float (1 dec.)	0.0 – 100.0	Nivel de llenado (%)
Q	float (2 dec.)	0.00 – 30.00	Caudal instantáneo (L/min)
B1	entero	0 / 1	Estado Bomba 1 (0=OFF, 1=ON)
B2	entero	0 / 1	Estado Bomba 2 (0=OFF, 1=ON)
SP	entero	0 – 100	Setpoint activo (%)
M	entero	0 / 1	Modo: 0=MANUAL, 1=AUTO
A	entero	0 / 1	Alarma: 1=Bomba ON sin caudal

Ejemplo de trama real:

N:75.3,Q:2.15,B1:1,B2:0,SP:80,M:1,A:0

Comandos ESP32 → Arduino

Comando	Efecto en Arduino	Ejemplo
B1:1 / B1:0	Bomba 1 ON / OFF (modo MANUAL)	B1:1
B2:1 / B2:0	Bomba 2 ON / OFF (modo MANUAL)	B2:0
SP:{valor}	Actualiza setpoint (0–100%)	SP:65
M:1 / M:0	Cambio a modo AUTO / MANUAL	M:1

Respuesta JSON del ESP32 (ruta GET /data):

{ "nivel": 75.3, "caudal": 2.15, "b1": 1, "b2": 0, "sp": 80, "modo": 1, "alarma": 0, "conexion": 1 }

4. INTERFAZ WEB HMI (EMBEDDED HTML/CSS/JS)

Red WiFi: ESP32-Llenado | **Contraseña:** control2026 | **URL de acceso:** http://192.168.4.1 | **Protocolo:** HTTP / REST (sin autenticación adicional)

La interfaz web está embebida directamente en el firmware del ESP32 como cadena de texto HTML/CSS/JavaScript almacenada en Flash (PROGMEM). No requiere servidor externo, tarjeta SD ni acceso a internet. El ESP32 opera en modo **Access Point**, creando su propia red WiFi a la que se conecta el dispositivo del operador.

Rutas del Servidor HTTP

Método	Ruta	Descripción
GET	/	Sirve la aplicación web completa (HTML+CSS+JS embebido)
GET	/data	Devuelve JSON con el estado actual del sistema
POST	/cmd	Recibe comandos de control (B1, B2, SP, M)

Elementos de la Interfaz

- **Indicador de nivel:** Barra animada azul (0–100%) con línea marcadora naranja para el setpoint.
- **Pantalla de caudal:** Valor en L/min actualizado en tiempo real.
- **Indicadores de estado de bombas:** ● (verde) = ON / ■ (gris) = OFF.
- **Botones de control manual:** ON/OFF para cada bomba (activos sólo en modo MANUAL).
- **Slider de setpoint:** Ajuste de 0–100%; envía comando SP:{val} al presionar.
- **Botón AUTO/MANUAL:** Cambia el modo de operación (azul=AUTO, naranja=MANUAL).
- **Estado del sistema:** Timestamp verde si hay conexión; rojo si no llegan datos.
- **Banner de alarma:** Aparece en rojo si la Bomba 1 está activa y el caudal es menor al mínimo (falla de bomba o tubería).
- **Tema visual:** Dark mode inspirado en GitHub, fuente monoespaciada, responsive.
- **Actualización automática:** El JavaScript consulta /data cada 1 segundo.

5. MODELADO MATEMÁTICO

5.1 Funciones de Transferencia

La planta se descompone en dos subsistemas en cascada: el sistema bomba+tubería (dinámica de primer orden con retardo de transporte) y el depósito (integrador puro).

Subsistema Bomba + Tubería (primer orden + retardo):

$$G_q(s) = K \cdot e^{(-Ls)} / (\tau_q \cdot s + 1)$$

Parámetro	Símbolo	Valor (experimental)	Descripción
Ganancia estática	K	2.5 L/min	Caudal máximo en régimen permanente
Constante de tiempo	τ_q	3.0 s	Tiempo de respuesta al 63%
Retardo de transporte	L	0.8 s	Retardo tuberías/válvulas

Subsistema Depósito (integrador puro):

$$G_h(s) = 1 / (A \cdot s)$$

Donde **A** = área transversal del depósito en L/cm. Valor experimental: A = 0.5 L/cm.

Planta completa (lazo abierto):

$$G_{total}(s) = K \cdot e^{(-Ls)} / [A \cdot s \cdot (\tau_q \cdot s + 1)]$$

Tipo de sistema: Tipo 1 (un integrador) → error de estado estacionario nulo frente a entrada escalón de referencia. **Polos de lazo abierto:** $\lambda_1 = 0$ (integrador), $\lambda_2 = -1/\tau_q = -0.333$ (estable). El sistema es marginalmente estable en lazo abierto.

5.2 Representación en Espacio de Estados

Se define un modelo de estados de orden 2 con las siguientes variables de estado:

$x_1 = h$: nivel de llenado (%)

$x_2 = q$: caudal instantáneo (L/min)

Las matrices del sistema en forma canónica son:

$$\dot{x} = A \cdot x + B \cdot u \quad y = C \cdot x + D \cdot u$$

$$A = \begin{bmatrix} 0 & 1/A \end{bmatrix} \quad B = \begin{bmatrix} 0 \end{bmatrix} \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad D = \begin{bmatrix} 0 \end{bmatrix} \quad \begin{bmatrix} 0 & -1/\tau_q \end{bmatrix} \quad \begin{bmatrix} K/\tau_q \end{bmatrix} \quad \begin{bmatrix} 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 \end{bmatrix}$$

Con los valores nominales (K=2.5, τ_q =3.0, A=0.5):

$$A = \begin{bmatrix} 0 & 2.0 \end{bmatrix} \quad B = \begin{bmatrix} 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -0.333 \end{bmatrix} \begin{bmatrix} 0.833 \end{bmatrix}$$

5.3 Análisis de Controlabilidad y Observabilidad

Matriz de controlabilidad:

$$M_c = [B \ AB] = \begin{bmatrix} 0 & 2.0 \cdot K/\tau_q \end{bmatrix}$$

$$\begin{bmatrix} K/\tau_q & -K/\tau_q^2 \end{bmatrix}$$

$$\det(M_c) = -K^2/\tau_q^2 \neq 0 \rightarrow \text{rango} = 2 \rightarrow \text{SISTEMA COMPLETAMENTE CONTROLABLE} \checkmark$$

Observabilidad: Ambas variables de estado (nivel y caudal) son medidas directamente por sensores físicos (HC-SR04 y GFS401), por lo que la matriz de observabilidad tiene rango completo. SISTEMA COMPLETAMENTE OBSERVABLE $\checkmark$

5.4 Análisis de Estabilidad

Aspecto	Resultado
Polos lazo abierto	$\lambda_1=0$ (integrador), $\lambda_2=-0.333$ (semiplano izq.)
Estabilidad L.A.	Marginalmente estable (polo en el origen)
Estabilidad con control histéresis	Oscilación acotada (bounded BIBO)
Estabilidad con LQR (propuesto)	Asintóticamente estable (todos los polos L.C. < 0)
Criterio de Routh (sin retardo)	Condiciones cumplidas para $K/\tau_q > 0$
Margen de fase (Bode)	A identificar experimentalmente
Rango de incertidumbre K	$K > 0 \rightarrow$ controlabilidad mantenida

6. ALGORITMO DE CONTROL

6.1 Control por Histéresis (Implementado en Firmware)

La estrategia de control implementada es un controlador ON/OFF con banda de histéresis (zona muerta). Este esquema evita el «chattering» (conmutaciones rápidas del relé) que ocurriría con un comparador puro. La lógica se evalúa cada $T_{CONTROL_MS} = 200\text{ ms}$.

Pseudocódigo del algoritmo de control:

```
si modo == AUTO: banda_inf = setpoint - HISTERESIS_PCT (ej. 80-5 = 75%) banda_sup =  
setpoint + HISTERESIS_PCT (ej. 80+5 = 85%) si nivel < banda_inf: setBomba(BOMBA1, ON) //  
Activar llenado sino si nivel > banda_sup: setBomba(BOMBA1, OFF) // Detener llenado //  
En [banda_inf, banda_sup]: no hacer nada (histéresis) si caudal < CAUDAL_MIN_ALARMA y  
bomba1==ON: alarma = true // Fallo de caudal
```

Parámetro	Valor Default	Efecto de Variación
SETPOINT_DEFAULT	80%	Nivel objetivo de llenado
HISTERESIS_PCT	5%	Mayor: menos ciclos, mayor error Menor: más ciclos, menor error
CAUDAL_MIN_ALARMA	0.5 L/min	Umbral de detección de fallo de bomba/tubería
T_CONTROL_MS	200 ms	Frecuencia de evaluación del lazo de control

Ventajas: Robusto, simple, no requiere tuning, previene rebose. **Limitaciones:** Oscilación permanente de $\pm 5\%$, no minimiza esfuerzo de control, respuesta más lenta que PID/LQR.

6.2 Control LQR Óptimo (Diseñado en MATLAB, Propuesto)

El regulador cuadrático lineal (LQR) minimiza el índice de desempeño:

$$J = \int_0^\infty [x^T \cdot Q \cdot x + u^T \cdot R \cdot u] dt$$

Parámetro	Valor	Criterio de Selección
$Q[1,1]$ (peso x_{nivel})	10	Penalizar error de nivel fuertemente
$Q[2,2]$ (peso x_{caudal})	1	Peso moderado en caudal
R (esfuerzo control)	1	Balance esfuerzo/desempeño
Ganancia óptima K_{lqr}	A calcular con MATLAB <code>lqr()</code>	Solución ecuación Riccati
Prealimentación N	$u = -K_{lqr} \cdot x + N \cdot r$	Elimina error en estado estacionario

El script MATLAB `simulacion_caudal.m` calcula automáticamente K_{lqr} , verifica la estabilidad del lazo cerrado, y compara la respuesta escalón del sistema con LQR versus el sistema en lazo abierto.

7. FIRMWARE: ARDUINO UNO (controlador_principal.ino)

Archivo: 04_Firmware/arduino_uno/controlador_principal/controlador_principal.ino | **Líneas:** 194 | **Configuración:** 04_Firmware/arduino_uno/include/config_arduino.h

El firmware del Arduino Uno implementa el lazo de control en tiempo real. Su estructura sigue el patrón de dos temporizadores independientes (timer-based polling) sin RTOS, ejecutando tareas a intervalos fijos:

Funciones Principales

Función	Período	Descripción
setup()	Una vez	Configura pines, inicializa comunicaciones, estado inicial
loop()	Continuo	Dispatcher principal, evalúa temporizadores
medirNivel()	200 ms	Promedia LECTURAS_PROMEDIO pulsos HC-SR04, convierte a %
medirCaudal()	1000 ms	Lee contador de pulsos INT0, convierte a L/min
ejecutarControl()	200 ms	Lógica de histéresis (modo AUTO)
setBomba(id, estado)	On demand	Actúa sobre relé (pin LOW/HIGH)
enviarEstado()	1000 ms	Formatea y envía trama serial al ESP32
leerComandos()	Continuo	Parser de comandos recibidos del ESP32
ISR(INT0_vect)	Por pulso	Rutina de interrupción: incrementa contador de pulsos

Detalles de Implementación Críticos

- **Promediado HC-SR04:** Se toman LECTURAS_PROMEDIO mediciones y se promedia para reducir ruido ultrasónico. Si alguna lectura excede el tiempo de timeout, se descarta.
- **Conteo de pulsos GFS401:** Usa la interrupción externa INT0 (flanco de bajada) para contar pulsos sin pérdida, incluso a frecuencias moderadas. El contador se limpia en cada ventana de 1 segundo.
- **Temporizadores no bloqueantes:** Se usa millis() para gestión de tiempo en lugar de delay(), permitiendo ejecución concurrente de múltiples tareas periódicas.
- **Lógica de histéresis:** La condición de histéresis garantiza que no se active ni desactive la bomba dentro de la banda [SP-H, SP+H], evitando oscilaciones rápidas.
- **Alarma de caudal:** Si la Bomba 1 lleva más de 2 ciclos ON y el caudal sigue por debajo de CAUDAL_MIN_ALARMA, se marca la bandera de alarma.

8. FIRMWARE: ESP32 (interfaz_wifi.ino)

Archivo: 04_Firmware/esp32/interfaz_wifi/interfaz_wifi.ino | **Líneas:** 331 | **Configuración:** 04_Firmware/esp32/include/config_esp32.h

Arquitectura del Firmware ESP32

El ESP32 implementa un servidor HTTP asíncrono (o síncrono, según versión) sobre WiFi en modo Access Point (AP). El bucle principal realiza polling de Serial2 para capturar tramas del Arduino y actualizar el estado global. Las peticiones HTTP son atendidas por handlers registrados.

Tarea / Handler	Disparador	Acción
setup()	Boot	Inicia AP WiFi, Serial2, registra rutas HTTP
loop()	Continuo	Polling Serial2, watchdog timeout Arduino
handleRoot()	GET /	Envía HTML+CSS+JS embebido al cliente
handleData()	GET /data	Serializa estado global a JSON
handleCmd()	POST /cmd	Parsea comando, reenvía al Arduino por Serial2
parseArduinoData()	Datos en Serial2	Extrae campos N,Q,B1,B2,SP,M,A
checkTimeout()	Cada iteración loop	Marca conexion=0 si silencio > 5000 ms

Parámetros de Configuración ESP32

Constante	Valor	Descripción
WIFI_SSID	"ESP32-Llenado"	Nombre de la red WiFi del AP
WIFI_PASS	"control2026"	Contraseña de acceso
AP_IP	192.168.4.1	Dirección IP del servidor web
SERIAL2_BAUD	9600	Velocidad comunicación con Arduino
SERIAL2_RX	GPIO 16	Pin de recepción
SERIAL2_TX	GPIO 17	Pin de transmisión
TIMEOUT_ARDUINO_MS	5000	Tiempo sin datos → 'Sin conexión'

9. SIMULACIONES

9.1 Simulación Python (simulacion_sistema_llenado.py)

Archivo: 05_Simulaciones/simulacion_sistema_llenado.py | **Líneas:** 615 | **Requiere:** Python 3.7+, numpy, matplotlib

El simulador es una réplica exacta del firmware del Arduino Uno más un modelo físico del sistema hidráulico. Esto permite validar la lógica de control antes de desplegar en hardware y obtener métricas de desempeño bajo condiciones controladas.

Modelo Físico Implementado

- **Dinámica del caudal (primer orden):** $dq/dt = (K \cdot u - q) / \tau_q$
- **Dinámica del nivel (integrador):** $dh/dt = q / A$
- **Ruido sensor HC-SR04:** $h_{medido} = h + \text{Gaussiano}(\mu=0, \sigma=0.4\%)$
- **Cuantización GFS401:** Error de cuantización por conteo entero de pulsos
- **Paso de integración:** $dt = 0.05 \text{ s}$ (Euler hacia adelante)

Tres Escenarios de Validación

#	Nombre	Condición Inicial	Eventos	Duración
1	Arranque desde cero	$h=0\%$, $q=0$, $SP=80\%$	Solo llenado automático hasta régimen	350 s
2	Tanque lleno – drenaje	$h=100\%$, $q=0$, $SP=40\%$	SP cambia, drenaje manual activado	120 s
3	Perturbación externa	$h=70\%$, $SP=70\%$	Drenaje súbito + recuperación + cambio	200 s

Salidas del Simulador

- Salida de consola con formato ANSI de colores (emula Monitor Serie del Arduino IDE).
- Reporte estadístico: tiempo de subida, sobrepaso, número de ciclos bomba, alarmas.
- Gráfica matplotlib de 4 paneles: nivel(%), caudal(L/min), estado bomba, alarmas.
- Imagen guardada como *simulacion_resultado.png* en 05_Simulaciones/.

9.2 Script MATLAB (simulacion_caudal.m)

Archivo: 05_Simulaciones/MATLAB_Simulink/simulacion_caudal.m | **Líneas:** 116 | **Requiere:** MATLAB R2023+ + Control System Toolbox

Sección	Contenido
1. Definición de TF	G_bomba(s), G_tanque(s), G_planta(s) con retardo

Sección	Contenido
2. Espacio de estados	Matrices A, B, C, D con parámetros nominales
3. Controlabilidad/Observabilidad	$\text{rank}(\text{ctrb}(A,B))$, $\text{rank}(\text{obsv}(A,C))$
4. Análisis L.A.	$\text{step}()$, $\text{bode}()$, $\text{rlocus}()$ de G_{planta}
5. Diseño LQR	$K_{\text{lqr}} = \text{lqr}(A, B, Q, R)$; polos lazo cerrado
6. Comparación L.C.	$\text{step}()$ lazo abierto vs cerrado con LQR

10. PARÁMETROS DE CALIBRACIÓN

Todos los parámetros ajustables por el usuario se centralizan en **04_Firmware/arduino_uno/include/config_arduino.h**. Modificar este archivo y recompilar permite adaptar el sistema a cualquier depósito o sensor sin tocar la lógica de control.

Constante	Tipo	Default	Rango	Descripción y Procedimiento de Ajuste
MAX_DIST_CM	float	30.0	10–50 cm	Distancia HC-SR04 con depósito vacío. Medir con cinta métrica.
LECTURAS_PROMEDIO	int	3	1–10	Mediciones a promediar. Aumentar si ruido es alto.
HISTERESIS_PCT	float	5.0	1–20 %	Semi-banda de histéresis. Compromiso ruido/oscilación.
SETPOINT_DEFAULT	int	80	0–100 %	Nivel objetivo al iniciar. Sobreescrito por slider web.
CAUDAL_MIN_ALARMA	float	0.5	0.1–2.0 L/min	Mínimo para alarma. Medir caudal con bomba recién arrancada.
FACTOR_CAL_GFS401	float	7.5	6–10 pulsos/L/min	Del datasheet. Calibrar con recipiente graduado.
T_CONTROL_MS	int	200	100–1000 ms	Período del lazo HC-SR04. Menor = más reactivo.
T_ENVIO_MS	int	1000	500–5000 ms	Período de transmisión serial y medición de caudal.

11. ESTRUCTURA DE DOCUMENTACIÓN DEL PROYECTO

Carpeta / Archivo	Estado	Contenido
README.md	✓ Completo	Visión general, inicio rápido, arquitectura
01_Documentacion/Especificaciones_Tecnicas/	✓ Completo	Especificaciones técnicas detalladas de todos los componentes
01_Documentacion/Manual_Usuario/manual_usuario.pdf	✓ Completo	Guía de operación para el usuario final (~80 páginas)
01_Documentacion/Manual_Usuario/manual_tecnico_completo.md	✓ Completo	Guía de ensamblaje y resolución de problemas (347 líneas)
01_Documentacion/Informe_Final/	■ Vacío	Informe académico final (pendiente)
01_Documentacion/Presentacion/	■ Vacío	Diapositivas de presentación (pendiente)
02_Modelado_Matematico/Funcion_de_Transferencia/	✓ Completo	Notas de modelado, procedimiento de identificación
02_Modelado_Matematico/Espacio_de_Estados/	✓ Completo	Formulación espacio de estados, diseño LQR
02_Modelado_Matematico/Analisis_de_Estabilidad/	■ Vacío	Análisis formal de estabilidad (pendiente)
03_Disenio_del_Sistema/Diagrama_de_Bloques/	■ Vacío	Diagramas de bloques del sistema (pendiente)
03_Disenio_del_Sistema/Esquematicos/	■ Vacío	Esquemas eléctricos (pendiente)
04_Firmware/arduino_uno/	✓ Completo	Firmware principal + header de configuración
04_Firmware/esp32/	✓ Completo	Servidor web WiFi + header de configuración
05_Simulaciones/simulacion_sistema_llenado.py	✓ Completo	Simulador Python de 615 líneas, 3 escenarios
05_Simulaciones/MATLAB_Simulink/simulacion_zafra01.m	✓ Completo	Script MATLAB 116 líneas, análisis completo
06_Resultados_y_Pruebas/Datos_Medicion/	✓ Plantilla	Formato CSV para registro experimental
07_Evidencias/	■ Vacío	Fotos y videos del montaje (pendiente)
08_Referencias/referencias.md	✓ Completo	Bibliografía y enlaces técnicos

12. LISTA DE MATERIALES Y COSTOS ESTIMADOS

Componente	Cantidad	Función	Costo USD (est.)
Arduino Uno R3 (ATmega328P)	1	Controlador principal	\$5–9
ESP32 DevKit v1	1	Servidor web WiFi	\$6–9
Sensor ultrasónico HC-SR04	1	Medición de nivel	\$2–4
Caudalímetro GFS401 (YF-S201)	1	Medición de caudal	\$3–5
Bomba DC sumergible 5V/12V	2	Llenado y drenaje	\$3–6 c/u
Módulo relé 2 canales (5V)	1	Actuación sobre bombas	\$1–2
Resistencia 10 kΩ	1	Divisor de tensión (nivel)	\$0.05
Resistencia 20 kΩ	1	Divisor de tensión (nivel)	\$0.05
Cables Dupont M-M y M-H	varios	Interconexiones	\$2–4
Protoboard 830 puntos	1	Montaje sin soldadura	\$2–4
Fuente 5V/2A (USB o DC)	1–2	Alimentación microcontroladores	\$3–6
Fuente 12V/1A (opcional)	1	Bombas de mayor potencia	\$3–6
Manguera flexible (Ø8mm)	1 m	Interconexión hidráulica	\$1–2
Abrazaderas/clips manguera	4	Fijación de conexiones	\$0.50
Contenedor/depósito 5–20 L	1	Planta hidráulica	\$3–8
TOTAL ESTIMADO	—	—	\$45–82 USD

13. GUÍA DE RESOLUCIÓN DE PROBLEMAS

Síntoma	Causa Probable	Solución
App web no carga	No conectado a WiFi del AP	Verificar conexión a 'ESP32-Llenado', contraseña 'control2026'
Nivel siempre 0% o 100%	MAX_DIST_CM incorrecto	Medir distancia vacío con cinta, actualizar config_arduino.h
Caudal siempre 0 L/min	GFS401 no en pin 2 o sin alimentación	Verificar cableado pin 2 (INT0), comprobar 5V al sensor
Bombas no responden	Relé no conectado o lógica invertida	Verificar IN1/IN2 al relé, relé es activo LOW (pin LOW=ON)
ESP32 no arranca tras upload	Error de compilación	Revisar errores en Arduino IDE, verificar selección de placa ESP32
Arduino se resetea al activar bomba	Pico de corriente en arranque	Usar fuente de alimentación separada para las bombas
'Sin conexión' en la web	Arduino no envía datos serial	Verificar SoftwareSerial pines 10/11, divisor de tensión
Nivel oscila mucho	Ruido excesivo en HC-SR04	Aumentar LECTURAS_PROMEDIO en config_arduino.h (hasta 5–10)
Alarma permanente con bomba ON	Caudal bajo por obstrucción	Verificar que la tubería no esté doblada o bloqueada
Setpoint no cambia desde web	Comando POST /cmd no llega	Verificar conexión WiFi activa y URL correcta 192.168.4.1

14. SECUENCIA DE OPERACIÓN

Arranque y Puesta en Marcha

1. Conectar fuente de alimentación al Arduino Uno (USB o DC 7–12V).
2. Conectar ESP32 (puede usar el mismo USB del computador o fuente independiente).
3. Esperar 5 segundos para que ambos microcontroladores inicialicen completamente.
4. Desde el teléfono o computador, buscar la red WiFi **ESP32-Llenado**.
5. Conectarse con la contraseña **control2026**.
6. Abrir navegador web y acceder a **http://192.168.4.1**.
7. Verificar que el estado muestre '**Conectado**' con timestamp en verde.
8. En modo **AUTO** (default): el sistema llena hasta el setpoint (80%) automáticamente.
9. Ajustar el setpoint con el slider si se desea otro nivel objetivo.
10. Para pruebas manuales: cambiar a modo **MANUAL** y controlar cada bomba individualmente.
11. Para drenar: activar Bomba 2 en modo MANUAL.

Diagrama de Estados del Sistema

```
Estado IDLE → (power ON) → INICIALIZANDO → (5s) → OPERACIÓN AUTO OPERACIÓN AUTO:
[nivel < SP-H] → Bomba1 ON (llenando) [nivel ∈ [SP-H, SP+H]] → Sin cambio (banda
muerta) [nivel > SP+H] → Bomba1 OFF (deteniendo) OPERACIÓN AUTO → (comando M:0) →
OPERACIÓN MANUAL OPERACIÓN MANUAL → (comando M:1) → OPERACIÓN AUTO CUALQUIER ESTADO →
(Bomba1 ON y Q < 0.5) → ALARMA (estado paralelo)
```

15. MEJORAS FUTURAS Y LÍNEAS DE DESARROLLO

Prioridad	Mejora	Beneficio Esperado
Alta	Implementar controlador PID	Respuesta más rápida, menor oscilación que histéresis
Alta	Desplegar controlador LQR diseñado en MATLAB	Control óptimo con mínimo esfuerzo de control
Alta	Control PWM de bombas (velocidad variable)	Elimina ON/OFF, control continuo del caudal
Media	Filtro de Kalman para fusión de sensores	Estimación de estado más robusta ante ruido
Media	Registro de datos en tarjeta SD	Trazabilidad temporal, análisis offline
Media	Comunicación MQTT a broker cloud	Monitoreo remoto desde cualquier ubicación
Media	Sensor de presión adicional	Segunda variable controlada, sistema MIMO
Baja	Integración con SCADA (Modbus/OPC-UA)	Compatibilidad con sistemas industriales
Baja	App móvil nativa (Flutter/React Native)	Mejor experiencia de usuario que web embebida
Investigación	Control predictivo MPC	Óptimo con restricciones explícitas de nivel/caudal

16. REFERENCIAS BIBLIOGRÁFICAS

- [1] Ogata, K. (2010). *Modern Control Engineering*, 5ª edición. Prentice Hall.
- [2] Nise, N. S. (2019). *Control Systems Engineering*, 8ª edición. Wiley.
- [3] Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2015). *Feedback Control of Dynamic Systems*, 7ª edición. Pearson.
- [4] Åström, K. J., & Wittenmark, B. (1997). *Computer-Controlled Systems: Theory and Design*. Prentice Hall.
- [5] Espressif Systems. (2023). *ESP32 Technical Reference Manual v5.x*. espressif.com.
- [6] Microchip Technology. (2022). *ATmega328P Datasheet*. microchip.com.
- [7] Arduino. (2024). *Arduino Reference – SoftwareSerial Library*. arduino.cc.
- [8] ARDUINO. (2024). *HC-SR04 Ultrasonic Sensor Datasheet*.
- [9] Seeed Studio. (2023). *YF-S201 Water Flow Sensor Manual*.
- [10] MATLAB. (2023). *Control System Toolbox – lqr() Reference*. MathWorks.
- [11] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.

FIN DEL DOCUMENTO

Sistema de Llenado de Flujo Continuo con Control Automático
Universidad Santiago Mariño – Teoría Moderna de Control

Autor: Neil Edickson Suarez Arevalo (NeilsAraquitec)
Co-Autor: Jose Fabien Salas Garcia

Generado el 01 de June de 2026, 18:32